

Sequence encoding of run histories

Paper 5 in a series on UK All-Weather racing outcomes

Patrick Gillen

2026-09-09

The models in this series score a horse on 26 features. Eight of them describe the horse's own run history, and papers 1 to 3 gave those eight as hand-built summaries: last time out's finishing position if it was in the first four, the position two runs back on the same rule, whether the horse had ever won, and a small going-affinity block. This paper replaces those eight with a gated recurrent encoder that reads the horse's twenty most recent prior runs directly, eight values a run, with the raw finishing position rather than a top-four code. The other eighteen features — official rating, weight, age, sex, headgear, draw, course, going, the layoff since the last run, and the trainer, jockey and sire strike rates — are unchanged, as are the races, the depth-three Plackett–Luce likelihood and the downstream scorer. On 2,183 held-out test races the encoder beats the hand-built features on every ranking measure, every interval excluding zero, and beats paper 3's gradient-boosted trees by more; Section 1 gives the figures. Three models now separate cleanly on a test split where papers 1 to 3 could not be told apart. Two further findings: a multi-layer perceptron on identical features beat a linear scorer, which paper 3's conclusion about function class did not anticipate; and embeddings — a short list of numbers the model learns for each trainer, jockey and sire — lost to the strike rates they replaced, both intervals excluding zero. The market still ranks better than any of these models. The single-bet win backtest improves to -12.2%, the best in the series, but the difference from paper 3 cannot be distinguished from zero.

Table of contents

1	The result	1
2	Data and scope	4
2.1	The race universe is paper 3's, unchanged	4
2.2	A validation slice, so the test split is scored once	4

2.3	The sequences	5
2.3.1	How much history there is	5
2.3.2	The values as the model reads them	6
2.4	One feature the sequences do not carry	6
3	Method	6
3.1	The objective is paper 2b's and paper 3's	6
3.2	A ladder, one change at a rung	7
3.3	Selection, and what it was allowed to see	8
3.4	The two models that reach the test split	8
4	The ladder, on the validation slice	8
4.1	Rung 1: the control that was meant to tie, and did not	9
4.2	Rung 2: embeddings for trainer, jockey and sire	10
4.2.1	The mechanism	10
4.3	Rung 3: the sequence encoder	11
4.4	The ladder in one table	12
5	The test split	13
5.1	Ranking	13
5.1.1	Against the market	14
5.2	Betting	15
5.2.1	Intervals on the returns	15
6	What the encoder is doing	16
6.1	How much history the gain needs	17
6.2	Whether it is re-encoding what it replaced	17
7	Conclusion	18
	Acknowledgments	19
8	Appendix A — The Plackett–Luce objective in torch	19
8.1	Verification against paper 3	19
8.2	The two-optimiser check	20
8.3	A bug a check on the loss value would have passed	20
9	Appendix B — Sequence construction	20
9.1	Schema	20
9.2	The cap, and why nothing is lost at it	21
9.3	Cleaning	21
9.3.1	The beaten-distance cap	22

10 Appendix C — Software and reproducibility	22
10.1 The series pattern	22
10.2 Fitting	23
References	23

1 The result

The models in this series score a horse on 26 features. Most of them describe the horse and the race rather than its recent form: the official rating, the weight carried, age, sex, headgear, the draw, the course, today’s going, the days since the horse last ran, and strike rates for the trainer, the jockey and the sire. Eight of the 26 describe the horse’s own run history, and those eight are the only ones this paper changes.

Four of the eight are position lags: last time out’s finishing position, recorded only if the horse finished in the first four and zero otherwise, with a companion indicator for the zero case, and the same pair for the run before that. The other four are an ever-won flag and three going-affinity terms.

This paper replaces those eight with the runs themselves. A gated recurrent encoder reads up to the twenty most recent prior runs, cross-surface, with eight values describing each run: the **raw** finishing position, how far the horse was beaten and whether that reading exists, the going, the class, the field size, the race distance, and the gap to the run before. The other eighteen features stay in both models, as do the races, the likelihood and the scorer.

Take Gordy Bee, running at 16 December 2010 with eight prior runs behind it. Its record, most recent first, is what the encoder reads:

Table 1: Gordy Bee’s eight prior runs, as the encoder receives them. Going is on the series’ ordinal scale; the days column is the gap back to the run before that one, so the earliest run in the career has no reading and carries the unavailable code -1 .

Run	Finish	Beaten (l)	Field	Class	Dist (f)	Going (ord.)	Days since previous
1 (last time)	9	6.62	14	5	10	4	23
2	4	4.25	14	5	8	4	13
3	6	17.50	12	5	10	7	431
4	3	3.38	19	5	7	4	19
5	9	13.50	10	5	8	4	24
6	8	15.25	16	4	8	4	17
7	2	4.00	6	5	10	2	326
8	6	5.25	17	4	7	5	-1

Paper 3 sees this record as four features: `pos_lag1_zero = 1`, `pos_lag1_nonzero = 0`, `pos_lag2_zero = 0`, `pos_lag2_nonzero = 4` — plus `has_wins = 0` and a going block of (5, 0.00, 0.00).

Last time out the horse finished 9th of 14, beaten 6.62 lengths. Because that is outside the first four, the position lag records it as zero and raises the indicator. A horse that finished last of twenty, beaten forty lengths, produces exactly the same pair. The run before was a fourth of 14, beaten 4.25 lengths, and is recorded as the single figure 4 — the same value a fourth beaten a short-head in a six-runner race would produce.

The rest of the table — the six earlier runs, the beaten distances, the class and distance of each race, the 431-day gap before the third-last run — is not in paper 3’s feature set at all.

Both models use the same MLP scorer and differ only in how the horse’s run history reaches it: the eight hand-built summary features in one, the encoder in the other. Both were fixed before any test race was scored. On 2,183 held-out test races:

Table 2: Encoder against hand-built features, held-out test split. Paired race-level bootstrap, B = 2000, 90% intervals.

Metric	En-coder	Hand-built	Difference	90% interval	Favours
P1_rank (higher better)	0.00433	0.00407	+2.586e-04	[+1.994e-04, +3.201e-04]	En-coder
Brier_place (lower better)	0.19741	0.20062	-3.204e-03	[-4.019e-03, -2.452e-03]	En-coder
pseudo-R ² (higher better)	0.06438	0.05379	+1.059e-02	[+8.220e-03, +1.300e-02]	En-coder

Every interval excludes zero. The same three comparisons against paper 3’s gradient-boosted trees (Chen and Guestrin 2016) are larger still, and the hand-built model itself beats paper 3 by a smaller margin, so the three models order strictly on all three measures:

$$\text{paper 3 (trees)} < \text{hand-built features (MLP)} < \text{sequence encoder (MLP)}$$

Nine contrasts, nine intervals excluding zero. That is worth setting beside what came before it. In papers 1 to 3 no model could be distinguished from another on this same test split: every paired interval in paper 3 against paper 2b contained zero, and the only contrast in that paper whose interval excluded zero was the market beating both models. Three models now separate cleanly.

Three things did not change. **The market still ranks better than any of them.** The encoder reaches P1_rank 0.00433 and Brier_place 0.19741 against the market baseline of

roughly 0.00543 and 0.1875 that papers 2b and 3 published. The gap narrows on both and closes on neither. The encoder and the market were not compared directly here; these are published figures set side by side.

No betting return improves in a way that can be distinguished from nothing. The single-bet-per-race backtest gives the encoder a win ROI of -12.19% against paper 3's -22.31% — the best in the series — but the difference is +10.11 points with a 90% interval of [-3.96, +23.34]. Eight of the nine ROI intervals contain zero. Every model loses money.

And the gain is not about long careers. Truncating the encoder to the five most recent runs keeps 96.6% of its advantage, so what it is doing is reading the recent runs properly rather than reading further back.

2 Data and scope

2.1 The race universe is paper 3's, unchanged

Nothing about which races are modelled changes in this paper. The scope is the series' own: UK All-Weather Flat handicaps in classes 2 to 5 at Kempton, Lingfield, Southwell and Wolverhampton, from 1 January 2006 to 14 October 2015, non-maiden, four to sixteen declared runners, complete cases on the model's features.

The rows are taken from the matrices paper 3 was fitted and scored on rather than re-derived from the filter rules, and row identity is asserted as a sorted set on (`race_id`, `runner_id`) rather than checked by counting races:

Table 3: Row identity against paper 3's frame.

Check	Value
rows	64389
row-identical to paper 3's frame on (<code>race_id</code> , <code>runner_id</code>)	TRUE
array dims	64389 x 20 x 8
seq_len within cap	20
no NA in the array	TRUE

The train/test split is the series' frozen chronological 70/30 at 2012-12-30, giving 5,022 training races (45,970 runner-rows) and 2,183 test races (18,419 runner-rows). No within-race information crosses the boundary.

2.2 A validation slice, so the test split is scored once

The models in this paper have hyperparameters — layer widths, learning rates, a hidden size, a dropout rate, an epoch count — and every one of them had to be chosen. Papers 2a and 2b had no such choices; paper 3 made them by cross-validation on the training split.

This paper carves a **validation slice** out of the training split by the same rule the series uses one level up: the date at which 70% of the training period’s races have been run, taken as a calendar date so it stays reproducible if the race universe changes. That date is 15 December 2010, giving 3,517 fitting races (Jan 2006 to Dec 2010) and 1,505 validation races (Dec 2010 to Dec 2012).

Every choice in this paper — features, architecture, hyperparameters, epoch count, which model goes forward — was made on that slice. The test split was scored once, at the end, after all of them were fixed.

2.3 The sequences

For each runner-race, the encoder receives the horse’s most recent prior runs, ordered most recent first, taken from the full cross-surface career history the series already extracts. The temporal boundary is the series’ own: strictly before the meeting date of the race being predicted.

Eight values describe each run:

value	what it carries
<code>finish_pos</code>	finishing position, raw , not capped at four
<code>beaten_dist</code>	cumulative lengths behind the winner
<code>beaten_missing</code>	indicator, distinguishing a real zero from an absent reading
<code>going_ordinal</code>	the going that day, on the series’ ordinal scale
<code>class</code>	race class on the 2006 scheme
<code>field_size</code>	runners in that race
<code>distance_furlongs</code>	race distance
<code>days_since_prev</code>	days from the run before it to that run

One convention matters for reading the tables below. **Unavailable readings take -1 , not zero**, because zero is a legitimate value for several of these — a winner is zero lengths behind the winner.

2.3.1 How much history there is

Training split only, 45,970 runner-rows:

Table 4: Prior-run counts on the training split. The capped row is what the encoder reads; the uncapped row is the horse’s true career length, which enters the model separately as a single feature.

Measure	Min	Q1	Median	Q3	P90	Max	Mean
sequence length (capped)	0	8	16	20	20	20	13.81
career runs prior (uncapped)	0	8	16	31	51	219	22.62

40.3% of training observations have careers longer than the twenty-run cap, and 91 (0.2%) have no prior run at all. Because truncation discards a horse’s oldest runs, the **untruncated** career length is carried alongside the sequence as a single feature, `career_runs_prior`, so nothing about career length is lost at the cap. A sequence of length zero is exactly “no prior run”, which is the fact the discarded indicators encoded.

2.3.2 The values as the model reads them

Each of the eight values was checked against its own scale, and out-of-range readings were handled in one of two ways. Training split, real slots only:

Table 5: The eight values describing each prior run, training split, over real (unpadded) slots.

Value	Real slots	Min	Median	Mean	Max	% zero
finish_pos	634,803	-1	5.00	5.391	35	0.00
beaten_dist	634,803	-1	4.25	7.619	100	13.59
beaten_missing	634,803	0	0.00	0.000	1	99.96
going_ordinal	634,803	0	4.00	3.982	7	0.46
class	634,803	-1	5.00	4.216	7	0.00
field_size	634,803	1	11.00	11.495	40	0.00
distance_furlongs	634,803	5	7.00	8.103	32	0.00
days_since_prev	634,803	-1	17.00	34.574	1000	0.00

A pre-2006 class code of 42 is not a class at all, so it is marked unavailable and takes -1 , as a finishing position of 99 does. A field size of 118 is on the right scale but implausible, so it is capped at 40 — as days since previous is capped at 1,000 and beaten distance at 100 lengths. Section 9 gives the counts.

2.4 One feature the sequences do not carry

The eight features the encoder replaces all summarise the horse’s history. A ninth that does the same, `days_LT0_log` — the gap from the horse’s most recent run to today’s race — is **kept**.

The sequences record the gap between one prior run and the one before it, not the gap from the last run to today’s race, because today’s race is not in the sequence. The encoder therefore keeps `days_LT0_log` as a separate feature, as it keeps `career_runs_prior`. Section 4 reports what it is worth: on the validation slice, restoring it moved the encoder from 5.741720 to 5.718496 per race, about 29% of the encoder’s whole advantage over the hand-built model.

3 Method

3.1 The objective is paper 2b’s and paper 3’s

The likelihood does not change in this paper. Each runner j in race r receives a score z_{rj} , and the probability of the observed finishing order down to depth k is the Plackett–Luce form (Plackett 1975; Luce 1959) that paper 2b’s exploded conditional logit fits and paper 3’s custom objective optimises:

$$\Pr(\text{top-}k \text{ order}) = \prod_{s=1}^S \frac{\exp(z_{r,(s)})}{\sum_{j \in R_{r,s}} \exp(z_{rj})}, \quad S = \min(k, J_r - 1),$$

where (s) is the horse that finished s -th and $R_{r,s}$ is the set still unplaced at stage s . Depth $k = 3$ throughout, as in papers 2b and 3. At $k = 1$ this is McFadden’s conditional logit (McFadden 1974), which is paper 1’s model.

What changes across the series is only the function producing z : a linear predictor in papers 1, 2a and 2b, a gradient-boosted tree ensemble in paper 3, and a neural scorer here. The loss is implemented directly in `{torch}` and verified against paper 3’s analytic gradient, so this paper descends the same surface paper 3 did; Section 8 gives the verification.

Weights are shared across horses. The scorer maps one runner’s features to one number, and runners are coupled only through the per-race denominator.

3.2 A ladder, one change at a rung

The paper is built as a ladder. Each rung changes exactly one thing about how a horse’s history reaches the scorer and is compared with the rung below it, so that a difference in ranking can be attributed to that change and not to a bundle of them.

rung	the one change	comparator
1	linear scorer → multi-layer perceptron, identical features	paper 2b’s linear predictor
2	three strike rates → embeddings for trainer, jockey and sire	rung 1
3	eight history summaries → sequence encoder	rung 1

The difference between two models is stated and asserted in code before either is fitted, rather than left to prose.

Two further rungs were specified and not run. Adding embeddings **alongside** the strike rates rather than in place of them would only test whether they help where the strike rate is already most reliable, since an embedding can only be learned for entities that appear often enough in the fitting data. A Deep Sets encoder over today’s field — a summary of the other runners rather than of the horse’s own past — would be fitted at race grain: 5,022 races against 45,970 runner-rows, an order of magnitude less data than every other rung, and a new architecture at the same time.

3.3 Selection, and what it was allowed to see

Nine configurations per model, declared in full before any of them was scored. Selection saw validation Plackett–Luce loss and nothing else: never a ranking metric, never a betting return, never a test row.

In every rung the best epoch arrived within about three passes over the training data, and the validation loss rose steadily afterwards. Early stopping therefore does real work, and it is done the same way for every model — the parameters kept are those from the best validation epoch.

3.4 The two models that reach the test split

The hand-built model scores 26 features. The encoder model scores 18 of those 26 — the eight history summaries of Section 2 are the ones removed — plus `career_runs_prior` and the sequence encoder’s output. The downstream scorer, the objective, the races, the validation slice and the seed are identical.

Both are refitted on the **full** training split — the validation slice folded back in — at the configuration the slice selected, then score the test split once. Both train for the same number of epochs, the epoch each selected on validation, and there is no per-epoch evaluation at that

stage: with no held-out set left, scoring the test split each epoch would put it inside the selection loop.

One consequence is worth stating plainly. The full training split is about 43% larger than the fitting partition used for selection, so an epoch is 43% more gradient steps than it was during selection. Both models take that change identically, so the contrast between them is unaffected by it.

4 The ladder, on the validation slice

Everything in this section is measured on the 1,505 validation races. No test row is involved.

4.1 Rung 1: the control that was meant to tie, and did not

Rung 1 exists to establish a baseline, not to find anything. The plan was to give a multi-layer perceptron exactly the features paper 2b’s linear predictor has, on the same likelihood and the same races, and confirm the two tie — so that any later gain from a sequence encoder could be attributed to the representation rather than to having switched to a neural network.

They do not tie.

Table 6: MLP against a linear scorer on identical 26 features and an equal selection budget, validation slice. Paired race-level bootstrap, $B = 2000$, 90% intervals.

Metric	MLP	Linear	Difference	90% interval	Favours
P1_rank (higher better)	0.00303	0.00299	+3.829e-05	[+1.042e-05, +6.579e-05]	MLP
Brier_place (lower better)	0.19700	0.19755	-5.430e-04	[-1.001e-03, -6.284e-05]	MLP

Both intervals exclude zero. The two models carry identical features, the same loss, the same slice and an equal budget of nine configurations at 200 epochs each.

Table 7 separates two changes. Paper 2b’s linear model has 17 features. Giving the same linear model nine more — the four going-affinity terms, the draw position and the four course indicators paper 2b lacks — improved it by 0.002999 per race. Switching that same 26-feature model from a linear scorer to an MLP improved it by a further 0.012710, about 4 times as much.

Table 7: Three fits on the validation slice: paper 2b’s linear model on 17 features, the same linear model on 26, and the MLP on those same 26.

Model	Features	Budget	Val PL loss	Gain on previous row
paper 2b, mlogit	17	closed form	5.814307	—
linear in torch	26	9 x 200	5.811308	0.002999
MLP	26	9 x 200	5.798597	0.012710

This sits against paper 3’s conclusion, and the paper should say so directly. Paper 3 replaced a linear predictor with a gradient-boosted tree ensemble on the same features and objective, found every paired interval containing zero, and read that as the function class not being the binding constraint. On the evidence in front of it that reading was sound, but the evidence was one function class. A different nonlinear model, on essentially the same information and the same objective, does better than the linear predictor here. What paper 3 established is that trees did not help; what it did not establish is that nonlinearity did not.

Two differences between the comparisons are worth keeping in view: paper 3’s contrast was scored on the test split while this one is on a validation slice, and paper 3’s trees route missing values natively where this model imputes them. Neither changes the direction of the finding.

4.2 Rung 2: embeddings for trainer, jockey and sire

An embedding is a short list of numbers the model learns for each trainer, jockey and sire, used in place of a single strike rate. Because the model fits those numbers itself, they can carry more than a win rate can — the supplementary note accompanying this paper gives the construction.

They do not update. Each trainer gets one vector, learned during fitting and fixed thereafter. A strike rate is recomputed before every race from the archive, so it tracks a trainer improving or declining across the period; an embedding cannot.

Rung 2 makes the swap. The three strike rates `trainerSR`, `jockeySR` and `sireSR` come out, leaving 23 features, and a four-number embedding of the trainer, of the jockey and of the sire goes in. The embeddings are learned only from the fitting rows, and an entity with fewer than twenty runs there is pooled into a shared bucket rather than given its own.

Table 8: Embeddings against the strike rates they replaced, validation slice.

Metric	Embed- dings	Strike rates	Differ- ence	90% interval	Favours
P1_rank (higher better)	0.00291	0.00303	-1.262e- 04	[-1.728e-04, -7.981e-05]	Strike rates

Table 8: Embeddings against the strike rates they replaced, validation slice.

Metric	Embed- dings	Strike rates	Differ- ence	90% interval	Favours
Brier_place (lower better)	0.19894	0.19700	+1.936e- 03	[+1.143e-03, +2.722e-03]	Strike rates

Both intervals exclude zero, against the embeddings. The swap costs 0.042520 per race on validation loss, and the separation is complete: the best of the nine configurations with embeddings, 5.841117, is worse than the worst of the nine with strike rates, 5.818877. The reading was fixed before the fit and it fired as *worse — report and stop, do not remediate*. No remediation was attempted.

4.2.1 The mechanism

A strike rate is arithmetic: the entity’s cumulative wins over runs at the latest meeting date before this race, computed over the whole 1.72M-row archive. It updates after every race the trainer has, and it is defined for trainers the model has never been fitted on.

An embedding is neither. It is one fixed vector per entity, learned only from the 32,296 in-scope fitting rows, and it exists only for entities that appear there often enough to be worth one.

Table 9: Which entities got their own embedding, frozen on the fitting partition.

Entity	Distinct in fitting	Given an embedding	% fitting rows pooled	% validation rows pooled	% validation rows unseen
trainer_id	657	296	6.34	15.54	6.35
jockey_id	541	185	4.38	19.77	9.19
sire_id	1,279	404	14.58	26.36	6.65

Under a chronological split that matters. About half the validation rows carry at least one entity in the shared bucket, against about a fifth of fitting rows, and on those rows the model reads a generic vector where the strike rate it replaced was a real number — `trainerSR`, `jockeySR` and `sireSR` are missing on no row of either partition.

Stated generally: **an embedding loses to a hand-built feature when the hand-built feature is computed from a wider information set than the model can be fitted on.**

4.3 Rung 3: the sequence encoder

Rung 3 removes the history summaries and adds the encoder. Nine configurations were declared, varying the four settings that belong to the encoder itself: the hidden size, the number of recurrent layers, the dropout rate and the learning rate, spread around a centre of hidden size 32, one layer, dropout 0.1 and a learning rate of 1e-3. The downstream scorer is held at rung 1’s selected architecture throughout, so the only architecture being searched is the encoder’s.

Table 10 gives the nine. Three things to take from it: dropout is the setting that matters, and its ordering is monotone — the highest setting is best and zero is second worst; capacity past a hidden size of 32 buys nothing; and the spread from best to worst configuration, 0.024525, is small against the 0.080102 the encoder gains over the hand-built model.

Table 10: The nine encoder configurations, validation slice, 60 epochs each. Selection on validation PL loss and nothing else.

Con-fig	Hid-den	Lay-ers	Dropout	Learning rate	Best val PL loss	Best epoch	Loss at epoch 60
5	32	1	0.3	1e-03	5.718496	3	6.011697
2	32	1	0.1	1e-03	5.729136	3	6.517491
3	64	1	0.1	1e-03	5.730836	1	7.826136
9	32	1	0.1	3e-04	5.731301	3	5.855294
8	32	1	0.1	3e-03	5.734502	6	8.824645
4	32	1	0.0	1e-03	5.735286	1	8.649143
6	32	2	0.1	1e-03	5.735824	3	6.551225
7	64	2	0.1	1e-03	5.737442	6	8.559757
1	16	1	0.1	1e-03	5.743021	3	6.280685

Refitting the selected configuration at 200 epochs reproduced its 60-epoch trace exactly — maximum absolute difference 0 — and did not improve on it. The optimum is at epoch 3 either way.

The encoder keeps `days_LTO_log`, for the reason given in Section 2. On the validation slice that term is worth 0.023224 per race:

Table 11: The layoff term in the encoder model, validation slice.

Metric	Layoff kept	Layoff dropped	Difference	90% interval	Favours
P1_rank (higher better)	0.00328	0.00321	+7.540e-05	[+4.235e-05, +1.082e-04]	Layoff kept
Brier_place (lower better)	0.19326	0.19435	-1.086e-03	[-1.589e-03, -5.488e-04]	Layoff kept

4.4 The ladder in one table

Table 12: Validation Plackett–Luce loss per race. Positive gain is better than rung 1.

Model	Val PL loss	Gain over rung 1
paper 2b, mlogit	5.814307	-0.015709
linear in torch, 26 terms	5.811308	-0.012710
rung 1 MLP, 26 terms	5.798597	—
rung 2 MLP, 23 terms + 3 embeddings	5.841117	-0.042520
rung 3 GRU, 18 terms + sequence encoder	5.741720	0.056878
rung 3b GRU, 19 terms + sequence encoder	5.718496	0.080102

The encoder is worth 0.080102 per race over the hand-built model — about 6.3 times the nonlinearity and 27 times the nine extra features — on **seven fewer** hand-built features. Which model went to the test split was decided on this column and recorded in the pipeline, not chosen by hand.

5 The test split

The test split was scored once, after every choice in Section 4 was fixed. Both models were refitted on the full training split, scored on 2,183 held-out races, and passed through paper 3’s own evaluation code — the same prediction builder, the same ranking metrics, the same backtest — with nothing differing between the models but the score vector. All three models are compared on an identical race universe.

5.1 Ranking

Table 13: Sequence encoder against the MLP on hand-built features, held-out test split. Paired race-level bootstrap over the common races, $B = 2000$, 90% intervals.

Metric	En-coder	Hand-built	Difference	90% interval	Favours
P1_rank (higher better)	0.00433	0.00407	+2.586e-04	[+1.994e-04, +3.201e-04]	En-coder
Brier_place (lower better)	0.19741	0.20062	-3.204e-03	[-4.019e-03, -2.452e-03]	En-coder
pseudo-R ² (higher better)	0.06438	0.05379	+1.059e-02	[+8.220e-03, +1.300e-02]	En-coder

Table 14: Sequence encoder against paper 3’s gradient-boosted trees, same races and same method.

Metric	En-coder	Paper 3	Difference	90% interval	Favours
P1_rank (higher better)	0.00433	0.00401	+3.135e-04	[+2.455e-04, +3.801e-04]	En-coder
Brier_place (lower better)	0.19741	0.20143	-4.019e-03	[-4.915e-03, -3.134e-03]	En-coder
pseudo-R ² (higher better)	0.06438	0.05145	+1.293e-02	[+1.016e-02, +1.562e-02]	En-coder

Table 15: The MLP on hand-built features against paper 3’s gradient-boosted trees, same races and same method.

Metric	Hand-built	Paper 3	Difference	90% interval	Favours
P1_rank (higher better)	0.00407	0.00401	+5.488e-05	[+1.735e-05, +9.178e-05]	Hand-built
Brier_place (lower better)	0.20062	0.20143	-8.149e-04	[-1.343e-03, -2.913e-04]	Hand-built
pseudo-R ² (higher better)	0.05379	0.05145	+2.334e-03	[+7.285e-04, +3.898e-03]	Hand-built

Nine intervals, nine exclusions of zero, every one favouring the newer model. The ordering is strict and identical on all three measures:

$$\text{paper 3 (trees)} < \text{MLP on hand-built features} < \text{sequence encoder}$$

Two things about the sizes. The encoder’s margin over paper 3 is larger than its margin over the hand-built model, which is what the ladder predicts: the hand-built model is itself ahead of paper 3. And that model’s own margin over paper 3 is roughly a fifth of the encoder’s, so most of the distance between paper 3 and the encoder comes from the representation rather than from the switch to a neural scorer.

The effect is larger on test than it was on validation. The model was finalised on the validation slice and then scored on the test races without further adjustment. On the validation slice the encoder beat the hand-built model by +1.774e-04 on P1_rank before the layoff term was restored; on test the corresponding figure is +2.586e-04. This is the same model measured on a different and larger sample, not an improvement in it.

5.1.1 Against the market

The encoder reaches P1_rank 0.00433 and Brier_place 0.19741. The over-round-adjusted starting price, on the baseline papers 2b and 3 published, reaches about 0.00543 and 0.1875.

The gap narrows on both measures and closes on neither. The encoder and the market were not compared directly here; these are published figures set side by side. Paper 4 already established the relevant result directly: against the settled starting price this series' models add nothing distinguishable, and their value sits in the interval between an early price and the off.

5.2 Betting

The single-bet-per-race rule is the series' own, and it selected nothing in this paper: no configuration, epoch, feature or model was chosen on a return. Among horses clearing the naive filter — model win probability above 0.15 and a model-to-market ratio above 1.3 — the rule backs the one with the highest model win probability. Win bets settle at starting price; place and each-way settle from the real-price payout tables.

Table 16: Single-bet-per-race backtest on the held-out test split.

Model	Bet	Bets	Wins	Profit	ROI	ROI less top winner
Encoder	win	1,064	131	-129.75	-12.19%	-13.52%
Hand-built	win	1,096	124	-212.17	-19.36%	-20.84%
Paper 3	win	1,133	124	-252.76	-22.31%	-23.74%
Encoder	place	1,064	475	-108.51	-10.20%	-10.70%
Hand-built	place	1,096	477	-82.39	-7.52%	-8.07%
Paper 3	place	1,133	471	-158.90	-14.02%	-14.56%
Encoder	each-way	1,064	475	-126.83	-5.96%	-6.76%
Hand-built	each-way	1,096	477	-183.65	-8.38%	-9.26%
Paper 3	each-way	1,133	471	-305.20	-13.47%	-14.33%

No model makes money. The encoder is the best of the three on win (-12.19%) and on each-way (-5.96%), and its win return is the best in the series to date.

It is also worst of the two paper-5 models on place, at -10.20% against the hand-built model's -7.52%, despite having the better Brier_place. The two measures are not in conflict: Brier_place scores calibration across every runner in the field, while the bet rule picks the single highest model win probability that clears a market filter, so they are computed on different quantities over different subsets. The result is recorded as it stands.

5.2.1 Intervals on the returns

Every other contrast in this paper carries an interval, and the returns should too. The per-race bet units were re-derived from the stored predictions, and checked to reproduce the published returns and bet counts exactly before anything was attached to them.

Table 17: Paired race-level bootstrap on single-bet ROI, over the common test races, $B = 2000$, 90% intervals. Differences in percentage points.

Comparison	Bet	First	Second	Difference	90% interval	Excludes zero
Encoder — Hand-built	win	- 12.19%	- 19.36%	+7.16	[-4.77, +18.23]	
Encoder — Hand-built	place	- 10.20%	- 7.52%	-2.68	[-8.61, +3.05]	
Encoder — Hand-built	each-way	- 5.96%	- 8.38%	+2.42	[-5.41, +9.76]	
Encoder — Paper 3	win	- 12.19%	- 22.31%	+10.11	[-3.96, +23.34]	
Encoder — Paper 3	place	- 10.20%	- 14.02%	+3.83	[-2.01, +10.11]	
Encoder — Paper 3	each-way	- 5.96%	- 13.47%	+7.51	[-1.27, +16.33]	
Hand-built — Paper 3	win	- 19.36%	- 22.31%	+2.95	[-7.01, +12.82]	
Hand-built — Paper 3	place	- 7.52%	- 14.02%	+6.51	[+1.65, +11.38]	yes
Hand-built — Paper 3	each-way	- 8.38%	- 13.47%	+5.09	[-1.19, +11.63]	

Eight of the nine intervals contain zero, and they are wide — roughly five to fourteen points either side. **No betting improvement in this paper is distinguishable from no improvement at all.**

One interval does exclude zero: the hand-built model’s place ROI is +6.51 points above paper 3’s, interval [+1.65, +11.38] — the first ROI interval in this series to exclude zero. The paper does not build on it: it is one of nine intervals with no multiplicity control and none pre-registered, it does not involve the paper’s model, and both models lose money.

6 What the encoder is doing

The result says the representation matters. It does not say what the encoder found. Two diagnostics narrow that, both on the validation slice, both descriptive.

6.1 How much history the gain needs

The selected configuration was refitted with the sequence truncated to the five, ten and fifteen most recent runs — same seed, same slice, only the number of runs the encoder can read changing.

Table 18: Sequence length against validation Plackett–Luce loss. The gain column is against the MLP on hand-built features.

Prior runs	Mean length	% at the cap	Best val PL loss	Gain over rung 1	Share of the full gain
5	4.78	88.1	5.721239	0.077358	96.6%
10	8.54	67.7	5.720594	0.078004	97.4%
15	11.46	53.0	5.719584	0.079013	98.6%
20	13.77	41.8	5.718496	0.080102	100.0%

Five runs keep 96.6% of the encoder’s advantage, and quadrupling the history available buys the rest. Paper 3 sees two prior runs. The advantage is therefore about reading the recent runs properly — the raw finishing position instead of a top-four code, with the beaten distance, going, class and field size attached to it — rather than about depth of history.

6.2 Whether it is re-encoding what it replaced

If the encoder were simply computing the discarded features for itself, its output would reconstruct them. Each removed feature was regressed on the encoder’s 32 output dimensions, on validation rows.

Table 19: Linear recovery of the removed features from the encoder’s output. Reference rows are features the encoder model kept, and was never required to encode.

Feature	R ²	n	Role
pos_lag1_zero	0.668	13,674	removed
pos_lag2_zero	0.428	13,674	removed
pos_lag1_nonzero	0.285	13,674	removed
going_sr_shrunk	0.213	13,611	removed

Table 19: Linear recovery of the removed features from the encoder’s output. Reference rows are features the encoder model kept, and was never required to encode.

Feature	R^2	n	Role
pos_lag2_nonzero	0.194	13,674	removed
going_runs_prior	0.183	13,648	removed
has_wins	0.140	13,674	removed
going_sr_delta	0.029	13,611	removed
or_relative	0.123	13,674	reference (kept)
days_LTO_log	0.099	13,674	reference (kept)
trainerSR	0.087	13,674	reference (kept)

The reference rows set the floor: features the encoder never had to represent sit at 0.087–0.123. Against that floor only one removed feature is substantially recovered — whether the horse finished outside the first four last time, at 0.668. Last time’s finishing position itself, which is the first of the eight values in the most recent slot, comes out at 0.285.

These are linear R^2 , so a low value means the feature is not *linearly* recoverable from those dimensions, not that the information is absent. What the table supports is the narrow claim that the encoder is not a linear re-encoding of the features it replaced.

7 Conclusion

Replacing eight of the model’s 26 features — the hand-built summaries of a horse’s own run history — with a sequence encoder that reads the runs themselves improves ranking on held-out data, by margins whose intervals exclude zero on every measure tested, and by more than the switch from trees to a neural scorer buys. After four papers in which no model could be distinguished from another on this test split, three now order strictly.

Two other things were found along the way. A multi-layer perceptron beat a linear scorer on identical features, which paper 3’s conclusion about function class did not anticipate and which that paper’s evidence did not cover. And embeddings for trainer, jockey and sire lost to the strike rates they replaced, cleanly and without remediation, because a hand-built feature computed from a wider information set than the model can be fitted on is a hard thing for an embedding to beat.

Two limitations bound the result. **or_relative** — the horse’s official rating relative to its race — stays in both models, and an official rating is the handicapper’s own summary of the same history, revised after every run from a wider base of races than this paper models. The encoder is competing for what is left after that, which makes the gain more interesting and also bounds it. And these models overfit quickly: the best epoch arrived within about three passes over the training data in every rung, which says how little 5,022 training races is for models of this size,

and is the strongest reason to be cautious about extending the architecture before extending the data.

All fitting was done on the CPU `libtorch` build in double precision. Results do not reproduce across backends, so every number here belongs to that one; Section 10 records what reproducing it requires.

What has not changed is the market. It ranks better than the encoder, as it ranked better than everything before it, and no betting return in this paper is distinguishable from no improvement at all. The representation of a horse’s history was a real constraint and is now a smaller one. It was not the constraint that separates these models from the price.

Acknowledgments

I have been interested in modelling horse racing since first acquiring the Smartform database more than a decade ago. The project has been picked up and put down many times, the obstacle almost always being the gap between an idea and a running implementation. What is different now is that working alongside Claude Code (Anthropic’s coding agent), with web-based Claude as a research companion, has compressed that gap dramatically. I am grateful to my brother, Michael Gillen, for the suggestion to build the model on All-Weather races — the foundation of the scope choices set out in Section 2. The design choices — dataset, scope, model, comparison, editorial line — are mine. The implementation, the bulk of the draft prose, and a great deal of the verification were Claude Code’s, under my direction; every output was reviewed and edited before it reached the paper.

8 Appendix A — The Plackett–Luce objective in torch

Paper 3 implements the depth- k Plackett–Luce likelihood as a custom XGBoost objective with an analytic gradient. This paper needs the same likelihood as a differentiable loss over variable-length races. It is verified against paper 3’s implementation rather than trusted.

8.1 Verification against paper 3

`scripts/verify_p5_pl_torch.R` asserts agreement with the function paper 3 was fitted with, on identical inputs: random scores at mixed field sizes; the edge cases, meaning two-runner races, races at exactly $k + 1$ runners, and ragged and single-race batches; depths $k = 1$ to 4; shift invariance; and score vectors spread widely enough that a naive `exp()` overflows.

Agreement is about $1e-9$ relative. The gradient matches finite differences to $2.9e-8$, and matches paper 3’s analytic gradient to $2.9e-8$, so this paper descends the same surface paper 3 did. The

check was run in two fresh R processes rather than twice within one, since state held at the process level can make a broken setting look verified on a second call.

8.2 The two-optimiser check

Agreement on a loss value does not establish that an optimiser reaches the same place. A linear scorer was therefore fitted in torch on paper 2b’s seventeen features and its validation loss compared with paper 2b’s own `{mlogit}` fit on the same rows — gradient descent against iteratively reweighted least squares, on the same objective and the same data.

Table 20: Two optimisers on the same objective and the same 17 features, validation Plackett–Luce loss per race.

Fit	Val PL loss	Best epoch
paper 2b, <code>{mlogit}</code> , closed form	5.814307	—
linear scorer, torch, Adam	5.813541	28

The two agree to 7.7e-04 per race, with the torch fit marginally the lower — which is what a first-order method run to convergence should produce against a closed-form fit with a different convergence tolerance.

8.3 A bug a check on the loss value would have passed

Entries that pad a race out to a common width must not contribute to its denominator. Writing that condition as `torch_where(mask, value, fallback)` returns the value of one branch but computes derivatives for both, and the discarded branch here contained `log 0`.

The forward loss was therefore exactly correct — agreeing with paper 3’s implementation to 1e-9 — while the gradient was `NaN`. A check on the loss value alone would have passed it and left every fit broken. The fix makes the denominator finite before the condition is applied rather than relying on the condition to remove an infinity, and the gate now asserts a finite gradient as well as a correct value.

9 Appendix B — Sequence construction

9.1 Schema

For each runner-race the builder produces a fixed-shape array plus two counts:

object	shape	what it is
<code>x</code>	$64,389 \times 20 \times 8$	prior runs $\times$ values per run, most recent first
<code>seq_len</code>	64,389	real runs in that row, capped
<code>career_runs_prior</code>	64,389	untruncated career length

Prior runs come from the series' cached cross-surface career history — every race code and every surface, not All-Weather alone — under the series' temporal rule: strictly before the meeting date of the race being predicted. Declared non-runners are dropped, matching the treatment the going features already give them: a declared non-runner did not run, so it is not a prior run.

9.2 The cap, and why nothing is lost at it

The array is capped at 20 runs. Career length is counted **before** the cap and carried separately as `career_runs_prior`, so a horse with fifty prior runs is not presented as one with twenty. On the training split 40.3% of observations exceed the cap.

The cap was set wide so that its value would not become another setting requiring selection. Table 18 confirms it never did: the encoder's advantage is essentially complete by five runs.

9.3 Cleaning

Two treatments, chosen by whether the reading is on its own scale.

Read as unavailable, taking -1 , when the value is not on the scale at all, so its magnitude would be meaningless:

- `class` outside 1–7. The 2006 restructuring replaced multi-digit codes whose meaning is not comparable with the current scheme, and those values are entirely pre-2006.
- `finish_pos` beyond 40. Values of 98 and 99 are sentinels, not placings.

Capped, when the value is on the scale but its exact magnitude stops being informative, so the ordering is worth keeping: `field_size` at 40, the largest legitimate British field; `days_since_prev` at 1,000, past which the exact gap carries nothing and against which the largest observed value of 37,918 days is plainly not a real layoff; and `beaten_dist` at 100 lengths.

-1 rather than 0 marks an unavailable reading because zero is legitimate for several of these values — a winner is zero lengths behind the winner, which is definitional rather than imputed.

9.3.1 The beaten-distance cap

`beaten_dist` is the only cap that touches an appreciable share of rows, so it is documented rather than asserted. Over 444,580 career-run rows, 208 (0.047%) have no reading at all.

Table 21: Rows above each beaten-distance threshold, whole career-run universe.

above	n	pct
30	31807	7.1577
50	14433	3.2480
75	7299	1.6425
100	4664	1.0496
150	1028	0.2313
200	293	0.0659
300	20	0.0045

The tail is real data, not a sentinel: the value is cumulative lengths behind the winner, and the long tail is jumps racing.

Table 22: Beaten distance by race type.

race_type	n	median	p99	max	n_over_100
All Weather Flat	160596	4.310	49.6200	349.50	638
Chase	5598	23.750	152.0150	430.00	378
Flat	241560	5.750	56.2500	306.75	1084
Hurdle	35846	22.000	171.8875	427.75	2525
National Hunt Flat	772	12.905	186.7575	394.31	39

A tailed-off horse in a hurdle race genuinely finishes hundreds of lengths adrift, so the reading is capped rather than discarded — the ordering still means “beaten a very long way”, and `-1` would misreport a real result as absent. Beyond 100 lengths the exact figure adds nothing. The cap also does the most for the scale the encoder sees: before it, the largest value sits about 23 standard deviations above the mean; after it, about 6.

10 Appendix C — Software and reproducibility

10.1 The series pattern

The analysis is a `{targets}` pipeline (Landau 2024) in R, with `{renv}` pinning package versions and one function per target. Paper 5 has its own pipeline script and its own store,

`_targets_p5.R` and `_targets_p5`, following paper 4’s arrangement, so nothing it does can invalidate or overwrite the published work of papers 1 to 4.

Upstream objects — the career history, the runner table, paper 3’s own fitting and test matrices, paper 2b’s fitted model — are read from the main store **read-only**, with their content hashes recorded in a fingerprint target. An upstream change therefore invalidates this paper’s downstream work rather than leaving it silently stale.

Every number in this document is read from a target at render time via `tar_read()` or `tar_load()`. None is transcribed. A figure with nothing behind it is a figure that can go stale without anyone noticing.

10.2 Fitting

Fitting uses `{torch}` (Falbel and Luraschi 2025) directly rather than through `{tidymodels}`. This is a documented exception of the same kind papers 1 and 2 make for `{mlogit}` and paper 3 makes for `{xgboost}` (Chen and Guestrin 2016): there is no interface in `{tidymodels}` for a custom Plackett–Luce objective over variable-length races with race-grouped reduction.

Table 23: Fitting environment, captured at fit time from inside the model-fitting functions.

torch	libtorch	R	Platform	Device	CUDA available	CPU threads
0.17.0	2.8.0	4.6.0	x86_64-w64- mingw32	cpu	FALSE	4

The lockfile pins the R packages but not the C++ runtime underneath `{torch}`, and the same R package version can sit on a CPU build or on any of several CUDA builds, which do not produce identical results. Everything here is fitted on the **CPU** build in double precision, so reproducing the environment requires setting `CUDA = "cpu"` before installing the backend; without it the installer probes for a CUDA toolkit and selects a different build. Because none of that appears in the lockfile, the backend is captured inside every fitting function and returned with the fitted model, then asserted against the pipeline’s own record, so a mixed-backend comparison fails loudly rather than being published.

References

Chen, Tianqi, and Carlos Guestrin. 2016. “XGBoost: A Scalable Tree Boosting System.” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16)* (San Francisco, CA), 785–94.

- Falbel, Daniel, and Javier Luraschi. 2025. *torch: Tensors and Neural Networks with GPU Acceleration*.
- Landau, William Michael. 2024. *The targets R Package User Manual*.
- Luce, R. Duncan. 1959. *Individual Choice Behavior: A Theoretical Analysis*. Wiley.
- McFadden, Daniel. 1974. “Conditional Logit Analysis of Qualitative Choice Behavior.” In *Frontiers in Econometrics*, edited by Paul Zarembka. Academic Press.
- Plackett, R. L. 1975. “The Analysis of Permutations.” *Journal of the Royal Statistical Society: Series C (Applied Statistics)* 24 (2): 193–202.